Experiment 2: Development of a Secure File Transfer Application

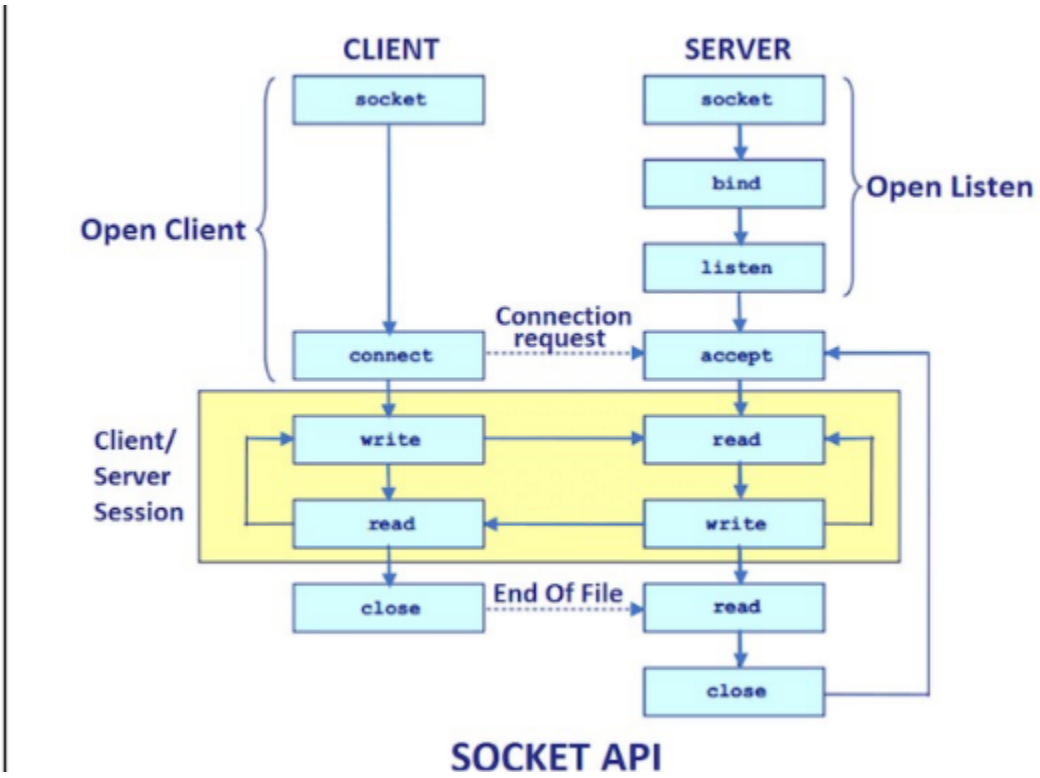
AIM:

To develop a **Secure File Transfer Application** using **Java Socket Programming** and **Java Swing GUI**, where files are securely transferred between a client and a server using the **Caesar Cipher** encryption and decryption technique.

PROCEDURE:

1. Create separate **Client** and **Server** applications using Java Socket Programming.
2. Design the client GUI using Java Swing with file list, text field, Upload, and Download buttons.
3. Establish a TCP connection using `ServerSocket` on the server and `Socket` on the client.
4. During **Upload**, read the client file, encrypt it using the Caesar Cipher, and send it to the server.
5. Store the received encrypted data in the server directory.
6. During **Download**, read the encrypted file from the server, send it to the client, and decrypt it at the client side.
7. Save the decrypted file in the client directory and verify successful file transfer.

FLOW CHART:



CODE:

TCPServer.java

```
import java.net.*;
import java.io.*;

public class TCPServer {
	public static void main(String args[]) {
		try {
			ServerSocket server = new ServerSocket(5000);

			System.out.println("Server Started...");
			System.out.println("Waiting for Client...");

			Socket socket = server.accept();
			System.out.println("Client Connected!");

			DataInputStream dis = new DataInputStream(socket.getInputStream());
			DataOutputStream dos = new
DataOutputStream(socket.getOutputStream());

			while (true) {
				String command = dis.readUTF();

				if (command.equals("UPLOAD")) {
					String filename = dis.readUTF();
					String fileData = dis.readUTF();

					FileWriter fw = new FileWriter(filename);
					fw.write(fileData);
					fw.close();

					System.out.println("Upload Completed : " + filename);
				} else if (command.equals("DOWNLOAD")) {
					String filename = dis.readUTF();

					BufferedReader br = new BufferedReader(new
FileReader(filename));
					StringBuilder data = new StringBuilder();
```

```
                                        String line;

                                        while ((line = br.readLine()) != null) {
                                                data.append(line).append("\n");
                                        }

                                        br.close();
                                        dos.writeUTF(data.toString());

                                        System.out.println("Download Completed : " + filename);
                                } else if (command.equals("EXIT")) {
                                        System.out.println("Client Disconnected");
                                        break;
                                }
                        }
                        socket.close();
                        server.close();
                } catch (Exception e) {
                        e.printStackTrace();
                }

        }
}
```

TCPClient.java

```
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;
import javax.swing.*;

public class TCPClient extends JFrame implements ActionListener {

   JLabel title, fileLabel;
   JTextField fileField;
   JTextArea fileList;
   JButton upload, download;

   Socket socket;
   DataInputStream dis;
```

```
DataOutputStream dos;

public TCPClient() {
    setTitle("TCP CLIENT");
    setSize(600, 450);
    setLayout(null);
    setDefaultCloseOperation(EXIT_ON_CLOSE);

    title = new JLabel("TCP CLIENT");
    title.setFont(new Font("Arial", Font.BOLD, 30));
    title.setBounds(190, 20, 300, 40);
    add(title);

    JLabel listLabel = new JLabel("Files in Client Directory");
    listLabel.setBounds(210, 80, 200, 20);
    add(listLabel);

    fileList = new JTextArea();
    JScrollPane sp = new JScrollPane(fileList);
    sp.setBounds(170, 105, 250, 100);
    add(sp);

    fileLabel = new JLabel("Enter File Name");
    fileLabel.setBounds(70, 240, 120, 25);
    add(fileLabel);

    fileField = new JTextField();
    fileField.setBounds(190, 240, 250, 25);
    add(fileField);

    upload = new JButton("Upload");
    upload.setBounds(120, 300, 120, 35);
    upload.addActionListener(this);
    add(upload);

    download = new JButton("Download");
    download.setBounds(320, 300, 120, 35);
    download.addActionListener(this);
    add(download);

    loadFiles();

    try {
        socket = new Socket("localhost", 5000);
```

```
            dis = new DataInputStream(socket.getInputStream());
            dos = new DataOutputStream(socket.getOutputStream());
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Server Not Running");
        }

        setVisible(true);
    }

    public void loadFiles() {
        File folder = new File(".");
        File files[] = folder.listFiles();

        fileList.setText("");

        for (File f : files) {
            if (f.isFile()) {
                fileList.append(f.getName() + "\n");
            }
        }
    }

    public void uploadFile() {
        try {
            String filename = fileField.getText();

            BufferedReader br = new BufferedReader(new FileReader(filename));

            StringBuilder sb = new StringBuilder();
            String line;

            while ((line = br.readLine()) != null) {
                sb.append(line).append("\n");
            }

            br.close();

            String encrypted = CaesarCipher.encrypt(sb.toString());

            dos.writeUTF("UPLOAD");
            dos.writeUTF(filename);
            dos.writeUTF(encrypted);

            JOptionPane.showMessageDialog(this, "Upload Successful");
```

```
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Upload Failed");
    }
}

public void downloadFile() {
    try {
        String filename = fileField.getText();

        dos.writeUTF("DOWNLOAD");
        dos.writeUTF(filename);

        String encrypted = dis.readUTF();

        String plain = CaesarCipher.decrypt(encrypted);

        FileWriter fw = new FileWriter(
                "Downloaded_" + filename);

        fw.write(plain);

        fw.close();

        JOptionPane.showMessageDialog(this, "Download Successful");

    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Download Failed");
    }
}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == upload) {
        uploadFile();
    }

    if (e.getSource() == download) {
        downloadFile();
    }
}

public static void main(String args[]) {
    new TCPClient();
```

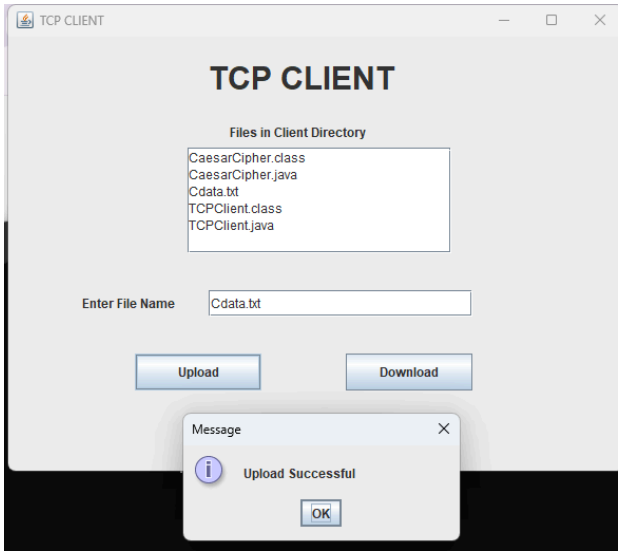
```
    }
}
```

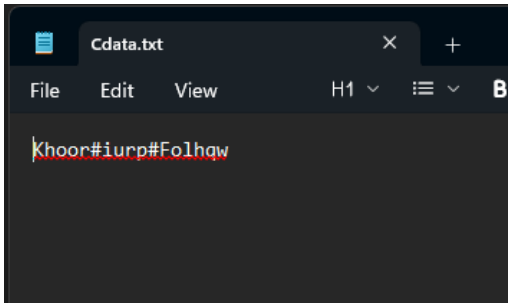
CaesarCipher.java

```
public class CaesarCipher{
        private static final int KEY=3;
        public static String encrypt(String text){
                StringBuilder encrypted=new StringBuilder();

                for(char ch: text.toCharArray()){
                        encrypted.append((char)((ch+KEY)%256));
                }
                return encrypted.toString();
        }
        public static String decrypt(String text){
                StringBuilder decrypted=new StringBuilder();

                for(char ch: text.toCharArray()){
                        int val=ch-KEY;
                        if(val<0)val+=256;
                        decrypted.append((char)val);
                }
                return decrypted.toString();
        }
}
```

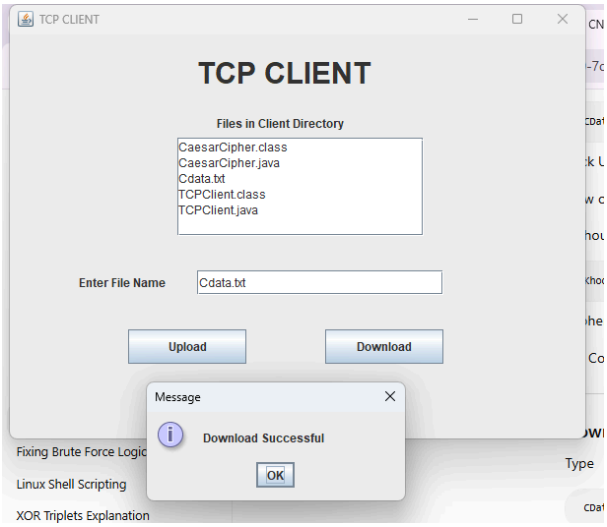
OUTPUT:

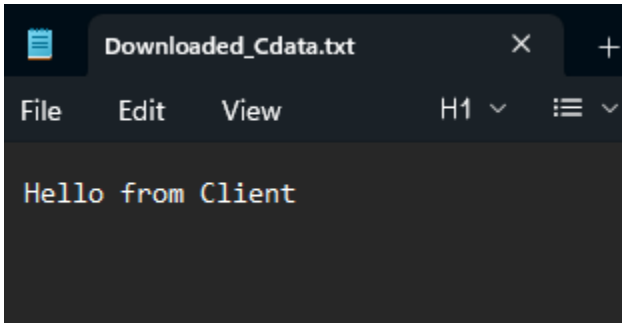
Encryption:





Decryption:






```
E:\128003214 CN\exp2\Server>java TCPServer.java
Server Started...
Waiting for Client...
Client Connected!
Upload Completed : Cdata.txt
Download Completed : Cdata.txt
```

RESULTS:

A secure file transfer application was successfully implemented using **Java TCP Socket Programming** and **Swing GUI**. The client was able to upload files after encryption using the Caesar Cipher, and the server successfully stored the encrypted file. During download, the encrypted file was transferred back to the client, decrypted successfully, and saved as the original plaintext file.

INFERENCE:

The experiment demonstrated secure file transfer using client-server communication over TCP sockets. It also illustrated the implementation of Caesar Cipher for encryption and decryption, ensuring confidentiality of the transmitted file while providing practical knowledge of Java networking and GUI programming.